

PERSONAL LEARNING OS

Master Architecture v3.0 — Antigravity Build Specification

Source preservation: This version retains the complete information from the supplied v2.0 architecture while incorporating the agreed engineering upgrades: strict local-first boundaries, safe file handling, a dedicated annotation layer, AI provider abstraction, RAG, Study State Engine, testing, recovery, performance and a controlled scalability path.

Target: Personal, single-user study application for Windows desktop and Android tablets. V1 requires no login and no cloud database for personal study data.

1. Master Technology Stack

Layer	Technology	Final decision / purpose
Frontend & UI	Flutter + Dart	Unified Windows + Android codebase; strong stylus, touch and CustomPainter/canvas support.
Database	SQLite	Single authoritative relational store for Documents → Pages → Annotations → Sessions → Tasks → Analytics.
Large files	Native OS filesystem	PDFs, audio, exports and caches remain files; SQLite stores metadata/references.
AI Server	Python + FastAPI	AI Gateway separating UI from model/provider logic.
Local AI	Ollama + Whisper + Piper/XTTS	Local LLM, speech-to-text and text-to-speech; privacy-first and offline-capable.
Document Intelligence	FAISS	Initial local vector index for RAG; retrieval layer remains replaceable.
Cloud AI	Gemini / Claude / OpenAI	Optional, on-demand providers through gateway adapters.
PDF/document conversion	Conversion engine + isolated temporary workspace	High-fidelity export without modifying originals; output validation required.

2. Product Scope — Nothing from the Original Vision Removed

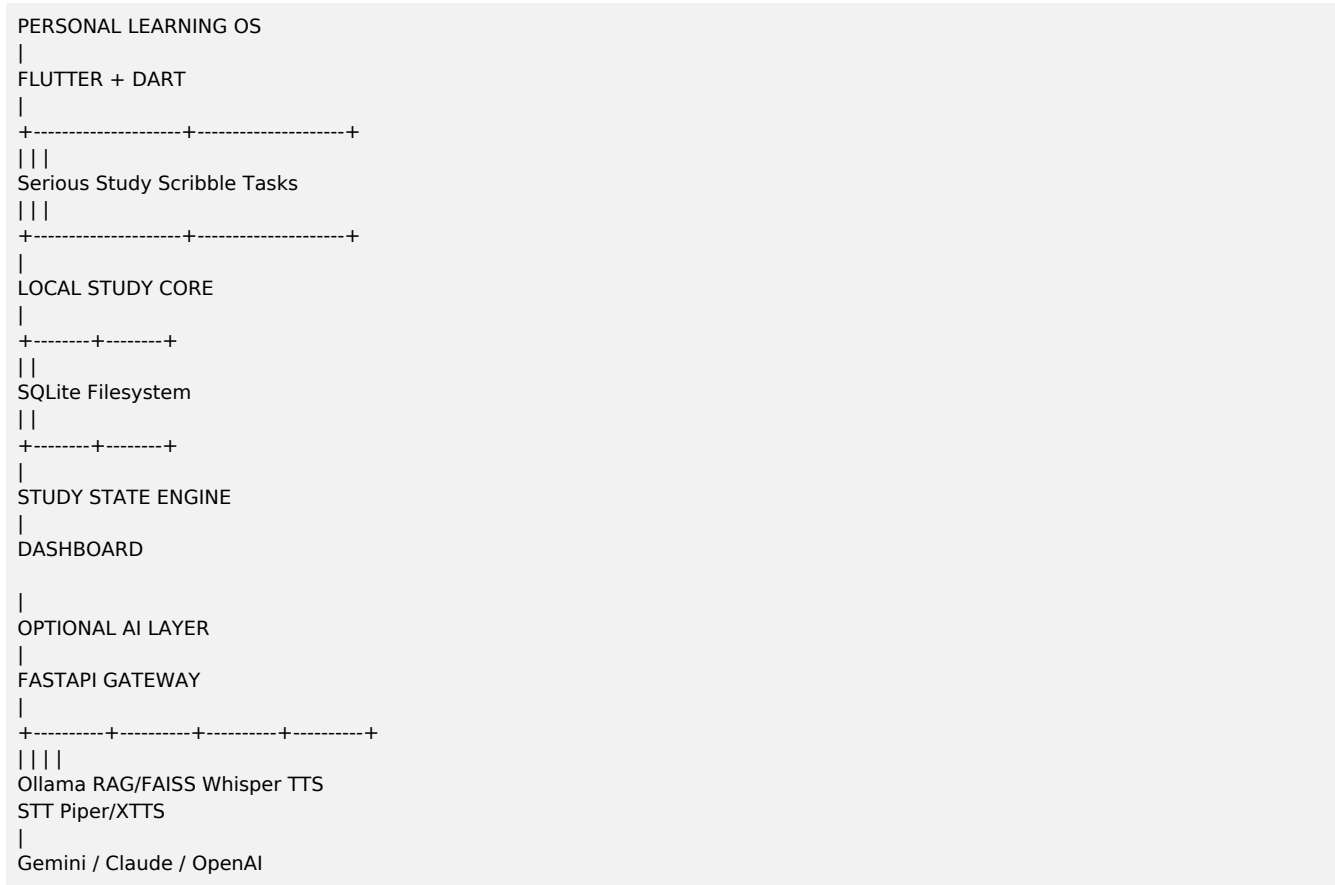
- Serious Study Mode: Open PDFs and other supported documents in a persistent workspace with pen tools, highlighters, notes, bookmarks, voice notes and study progress.
- Scribble Mode: Fast temporary study canvas for rough writing, calculations and diagrams. It may be discarded or explicitly saved/exported.
- AI Voice / Study Companion: Human-like voice interaction for motivation, mood support, competitiveness, urgency, explanations and study assistance, using local/open-source models where configured.
- Document Converter: High-precision document conversion/export for safer transport and file delivery.
- External AI: Connect on demand to Gemini, Claude and ChatGPT/OpenAI so selected app information can be used by those assistants.
- Task Manager: Checklists, generated study tasks, progress and motivational tracking.
- Growth Dashboard: Charts and analytics recording study growth, consistency, tasks, outcomes and topic-level performance.
- Storage model: Personal V1 uses system/local storage. No user-data backend and no login are required initially.
- Windows support: Windows is a first-class target, alongside Android tablets.

3. Non-Negotiable Architecture Rules

- Original documents are immutable. Never annotate by rewriting the source file.
- Study annotations form a separate layer and are persisted as structured data.

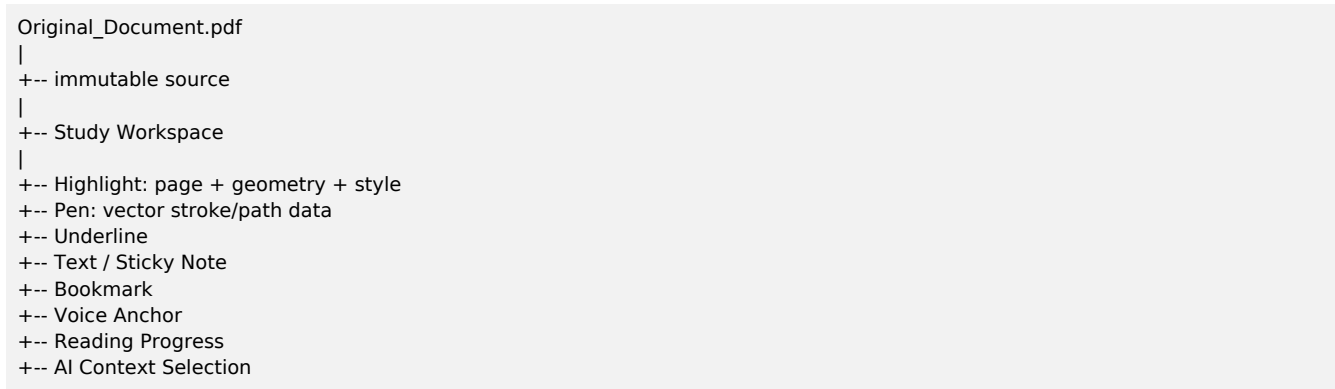
- SQLite is the only authoritative structured-data database in V1. Do not introduce Isar as a second source of truth.
- Opening documents, annotating, scribbling, notes and tasks must not require an AI server.
- AI is an enhancement layer, not a dependency of the Study Core.
- Flutter must not contain provider API keys or provider-specific business logic.
- Remote AI receives only selected/required context, never the user's entire local storage automatically.
- Heavy extraction, conversion, embedding, inference and indexing must not block the UI thread.
- Generated exports must be created in a temporary workspace, validated, then moved to Exports.
- Every major engine must have an interface so its implementation can be replaced later.

4. Master System Architecture

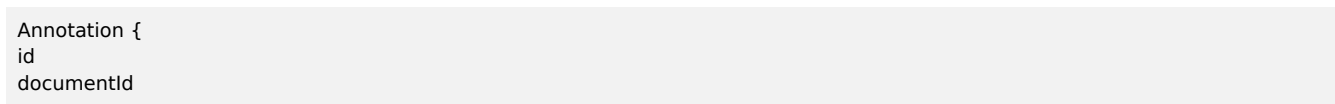


5. Core Engine A — Document & Annotation

The original v2.0 rule is retained: the source PDF is never modified. User interactions are overlaid through a separate study layer. The upgrade makes the storage contract explicit.



Annotation data contract



```

page
type
geometry
style
content
createdAt
updatedAt
}

```

Vector strokes are preferred over screenshot-only storage so pen marks remain scalable, editable and suitable for undo/redo.

6. Core Engine B — Scribble

```

Scribble
├── Pen
├── Highlighter
├── Eraser
├── Undo / Redo
├── Clear
├── Temporary Auto-save
├── Save Permanently
└── Export

```

Default: temporary storage.
Permanent save: explicit user action.

Scribble must feel immediate and independent of the document reader. A saved scribble may later become a permanent study note/export.

7. Core Engine C — AI Gateway & Multi-Model Orchestration

The supplied v2.0 design routes intelligence through FastAPI and prevents the Flutter client from hardcoding model logic. That principle is retained, with provider adapters made explicit.

```

Flutter Client
|
REST / WebSockets
v
FastAPI AI Gateway
|
+-- LocalLLMProvider -> Ollama
+-- STTProvider -> Whisper
+-- TTSPProvider -> Piper / XTTS
+-- GeminiProvider
+-- ClaudeProvider
+-- OpenAIProvider
+-- RAGService -> FAISS

```

The client requests capabilities such as explain, summarize, quiz, evaluate, generateTasks, retrieveContext or speak. It does not request provider-specific implementation details.

8. Core Engine D — RAG / Document Intelligence

The supplied v2.0 RAG sequence is retained exactly in concept: PDF → extraction → chunking → embeddings → FAISS → retrieve relevant context → send context + query to LLM → answer.

1. User imports document
2. Text extraction
3. Cleaning / normalization
4. Chunking
5. Embedding generation
6. Store embeddings in local FAISS index
7. User asks a document question
8. Retrieve relevant chunks
9. Build selected context
10. Send context + query to chosen LLM
11. Return answer

Scope retrieval by document and, where possible, current page/selection. This reduces irrelevant context and model cost.

9. Core Engine E — Voice AI

```

Microphone
|
Whisper STT
|
Study Context + User Request
|
AI Gateway / Local LLM / External Provider
|
Response Text
|
Piper / XTTS
|
Speaker

```

Voice is a study companion rather than only a narrator: motivation, focus prompts, explanations, question/answer interaction and study guidance are supported. Model implementations remain replaceable.

10. Core Engine F — Document Conversion & Export Safety

```

Input
|
Detect + Validate
|
Create temporary working copy
|
Convert
|
Validate output
|
Write final export
|
/Exports/

```

- Never overwrite the source document.
- Keep failed/partial conversion files out of the final export directory.
- Use reference documents for conversion regression tests.
- Treat conversion as an isolated service so the UI remains responsive.
- LibreOffice may be used as a conversion engine where supported, but the engine must sit behind a conversion interface.

11. Core Engine G — Tasks & Checklists

```

Subject / Topic / Document
|
User or AI-generated plan
|
Study Tasks
|
Read → Understand → Practice → Revise → Test
|
Completion
|
Analytics

```

Tasks remain editable. Each task can include status, priority, estimated duration, due date, topic, source document and completion timestamp.

12. Core Engine H — Study State Engine

The system should measure learning context, not only app activity.

```

Study Session
├── Subject / Topic
├── Document ID
├── Pages Covered
├── Duration
├── Focus Interruptions
├── Tasks Completed
├── Notes Created
├── Questions Answered
├── Quiz Accuracy
└── Difficulty / Confidence

```

This is the data foundation for intelligent dashboard recommendations and weak-topic detection.

13. Dashboard / Growth Intelligence

- Daily / weekly / monthly study time
- Consistency and streaks
- Task completion
- Document/page progress
- Quiz accuracy
- Topic confidence
- Revision frequency
- Strong and weak topics
- Study session outcomes
- AI/voice study activity

The dashboard must distinguish time spent from actual learning outcomes. High study time with low accuracy should be visible as a different state from successful mastery.

14. Local Storage Contract

```
/LearningOS_Data/  
├─ database.sqlite  
├─ /Original_PDFs/  
├─ /Study_Data/  
├─ /Scribbles_Temp/  
├─ /Voice_Notes/  
├─ /AI_Cache/  
└─ /Exports/
```

This corrects the ambiguity in the supplied v2.0 folder description: the database is authoritative and generated exports are kept separately. Original PDFs remain untouched.

15. Database Model

```
documents  
├─ document_versions  
├─ bookmarks  
├─ annotations  
├─ notes  
└─ voice_notes  
  
study_sessions  
├─ session_events  
└─ focus_intervals  
  
tasks  
├─ task_items  
└─ task_dependencies  
  
topics  
├─ topic_progress  
└─ topic_reviews  
  
quizzes  
├─ questions  
├─ answers  
└─ results  
  
daily_statistics  
ai_sessions  
settings
```

16. Offline-First Contract

NO INTERNET REQUIRED:

- ✓ Open/read documents
- ✓ Serious Study annotations
- ✓ Scribble
- ✓ Notes
- ✓ Tasks/checklists
- ✓ Existing dashboard data
- ✓ Saved voice notes
- ✓ Local AI when models are installed

OPTIONAL INTERNET:

- Gemini
- Claude
- OpenAI
- Remote model services
- Future sync

17. Security & Privacy

- V1 has no login because the product is personal.
- Remote AI is opt-in and context-limited.
- Provider API keys are kept out of the Flutter client.
- HTTPS is required for remote AI traffic.
- Do not log document contents or private notes by default.
- If client-side secrets become necessary later, use OS secure credential storage.
- Future cloud synchronization must be designed as an explicit opt-in feature.

18. Error Recovery & Data Integrity

- Use SQLite transactions for related multi-record updates.
- Recover or discard incomplete temporary conversion outputs safely.
- Persist annotation changes incrementally.
- Prevent a failed export from replacing a valid export.
- Handle missing source files gracefully.
- Handle unavailable AI/TTS/STT services with a clear non-AI fallback.
- Detect low-storage conditions before large exports when possible.
- Provide a future backup/restore mechanism for database + study assets.

19. Performance Requirements

- Lazy-load PDF pages and thumbnails.
- Do not load an entire large document into memory unnecessarily.
- Do not block the UI with OCR/extraction, conversion, embeddings or inference.
- Cache thumbnails, embeddings and safe reusable AI results.
- Keep binary documents/audio outside SQLite.
- Batch non-critical analytics writes.
- Stream AI responses where supported.
- Test large PDFs and high annotation counts before release.

20. Testing Strategy

- Unit: annotation geometry, repository logic, task rules, analytics calculations and AI request validation.
- Integration: SQLite + filesystem, conversion pipeline, RAG retrieval and AI gateway.
- Golden-file: known PDFs/documents and expected conversion outputs.

- Recovery: interrupted writes, failed conversion, unavailable AI and app restart.
- Performance: large PDFs, thousands of annotations, long histories and large indexes.
- Platform: Windows and Android tablet touch/stylus, filesystem and audio behavior.

21. Project Structure for Antigravity

```
personal-learning-os/  
├── apps/  
│   ├── client/ # Flutter Windows + Android  
│   └── ai_server/ # FastAPI AI Gateway  
├── packages/  
│   ├── core/  
│   ├── document_engine/  
│   ├── annotation_engine/  
│   ├── study_engine/  
│   ├── ai_core/  
│   ├── rag_engine/  
│   ├── voice_engine/  
│   ├── task_engine/  
│   └── analytics_engine/  
├── tests/  
├── docs/  
└── scripts/  
  
client/lib/  
├── core/  
│   ├── database/  
│   ├── filesystem/  
│   ├── routing/  
│   ├── theme/  
│   └── error_handling/  
├── features/  
│   ├── reader/  
│   ├── serious_study/  
│   ├── scribble/  
│   ├── notes/  
│   ├── voice/  
│   ├── ai/  
│   ├── tasks/  
│   ├── dashboard/  
└── shared/
```

22. Development Order

1	Foundation	Flutter targets, app shell, SQLite, filesystem, settings, logging and error handling.
2	Document Reader	Import, validation, PDF viewing, recent files, bookmarks.
3	Serious Study	Annotation engine, pen, highlighter, notes, undo/redo, persistence.
4	Scribble	Fast canvas, temporary storage, save/export.
5	Conversion	Safe conversion workspace, validation and exports.
6	AI Gateway	FastAPI contracts, Ollama adapter and core AI actions.
7	RAG	Extraction, chunking, embeddings, FAISS retrieval and document Q&A.;
8	Voice	Whisper, TTS, voice notes and voice study companion.
9	Tasks	Checklist generation, plans, quiz and task analytics.
10	Study State	Session tracking, topic state, outcomes and revision logic.
11	Dashboard	Charts, weak-topic analysis and recommendations.
12	External AI	Gemini, Claude and OpenAI adapters with explicit context sharing.
13	Hardening	Testing, recovery, performance, packaging, backup/restore and release.

23. Scalability Path

```
V1 — PERSONAL / LOCAL  
Flutter + SQLite + Files + FastAPI + Ollama  
|
```

```
v
V2 — HYBRID AI
Provider adapters + RAG + model routing
|
v
V3 — OPTIONAL CLOUD
Authentication + encrypted sync + cloud storage
|
v
V4 — MULTI-USER
User service + object storage + scalable workers
|
v
V5 — PLATFORM
Multi-device sync + teams + advanced model routing + analytics services
```

Do not build cloud authentication, cloud databases or microservices into V1. Preserve interfaces now so those capabilities can be added later without rewriting the Study Core.

24. Final Build Rules for Antigravity

- Treat this document as the master source of architectural truth.
- Do not add a framework or database merely because it is popular.
- Do not create duplicate persistence layers.
- Do not put AI logic directly inside Flutter widgets.
- Do not put API keys in source code or the packaged client.
- Do not modify imported originals.
- Do not make core study operations depend on network availability.
- Implement interfaces before provider-specific implementations.
- Prefer simple modules with explicit responsibilities.
- When a requirement conflicts with this specification, preserve data integrity, offline functionality and modularity first.

25. Final Architecture Decision

Locked V1: Flutter + Dart; Windows + Android tablets; SQLite; native filesystem; FastAPI AI Gateway; Ollama; Whisper; Piper/XTTS; FAISS; optional Gemini/Claude/OpenAI adapters; safe document conversion; Serious Study; Scribble; Tasks; Voice; RAG; Study State Engine; Growth Dashboard.

Core principle: AI enhances the Personal Learning OS. The Personal Learning OS must remain functional without AI or internet access.

Scalability principle: Keep the Study Core local and stable; scale the AI and optional cloud layers independently.